

JSP - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

JSP — MASTER NOTES	5
A Complete Reference Book for Interviews, Production Work & System Understanding	5
TOPIC 1: JSP LIFECYCLE	5
1. Introduction	5
2. The JSP Lifecycle — Translation, Compilation, Execution	5
3. Diagram — What Actually Happens to Your .jsp File	5
4. Real Life Analogy	5
5. First Request vs Subsequent Requests	6
6. Edge Cases	6
7. Common Mistakes	6
8. Frequently Asked Interview Questions	6
TOPIC 2: JSP DIRECTIVES	6
1. Introduction	7
2. The Three Directive Types	7
3. page Directive — Common Attributes	7
4. Static Include vs Dynamic Include — A Frequently Confused Pair	7
5. Frequently Asked Interview Questions	7
TOPIC 3: SCRIPTLETS	8
1. Introduction	8
2. The Three Scripting Elements	8
3. Why Scriptlets Are Discouraged (Critical Modern Understanding)	8
4. Real Life Analogy	8
5. Frequently Asked Interview Questions	8
TOPIC 4: EXPRESSION LANGUAGE (EL)	9
1. Introduction	9
2. Syntax and Scope Resolution	9

3. EL Operators	9
4. Code Example	9
5. \${} (EL, JSP standard) vs #{} (JSF-specific) — A Common Confusion	10
6. Edge Cases	10
7. Common Mistakes	10
8. Best Practices	10
9. Frequently Asked Interview Questions	10

TOPIC 5: JSTL (JSP STANDARD TAG LIBRARY) 11

1. Introduction	11
2. The Core (c) Tag Library — Most Common Tags	11
3. Comparison — Scriptlet Loop vs JSTL Loop (Direct Before/After)	11
4. Other JSTL Libraries (Beyond Core)	12
5. Real Life Analogy	12
6. Edge Cases	12
7. Common Mistakes	12
8. Best Practices	12
9. Frequently Asked Interview Questions	13

TOPIC 6: CUSTOM TAGS 13

1. Introduction	13
2. Building a Simple Custom Tag	13
3. Real Life Analogy	14
4. Custom Tags vs JSTL — When to Build Your Own	14
5. Frequently Asked Interview Questions	14

TOPIC 7: MVC WITH JSP 14

1. Introduction	15
2. Model 1 vs Model 2 Architecture	15
3. Full Code Example — A Complete Model 2 (MVC) Flow	15
4. Real Life Analogy	15
5. Edge Cases	16
6. Common Mistakes	16

7. Best Practices	16
8. Production Usage	16
9. Frequently Asked Interview Questions	16
10. Revision Notes (Entire JSP Section Consolidated)	17
11. Homework (Covers Entire JSP Section)	17
FINAL SUMMARY — JSP COMPLETE	17

JSP — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: JSP LIFECYCLE

1. Introduction

What is it? JSP (JavaServer Pages) is a technology that lets you embed Java logic directly inside HTML-like template files — under the hood, EVERY JSP page is automatically TRANSLATED into a regular Servlet (extending `HttpJspBase`, itself extending `HttpServlet`) by the container, then compiled and executed exactly like any other Servlet.

Why introduced: Writing HTML output using raw Servlets (`response.getWriter().println("<html>...")` for every single line) is extremely awkward for anything beyond trivial pages. JSP flips the model — you write HTML-like markup NATURALLY, embedding small snippets of Java logic only where actually needed, and the container handles the translation into an executable Servlet automatically.

2. The JSP Lifecycle — Translation, Compilation, Execution

1. TRANSLATION -> the container translates the .jsp file into an EQUIVALENT .java Servlet source file (happens ONCE, or when the .jsp file changes)
2. COMPILATION -> that generated .java file is compiled into a .class file (also happens ONCE, or on change)
3. INSTANTIATION -> the container instantiates the generated Servlet class (exactly like the Servlet Lifecycle - Topic 1 of Servlets)
4. `jspInit()` -> called ONCE, equivalent to a Servlet's `init()`
5. `_jspService()` -> called for EVERY request (equivalent to a Servlet's `service()`) - THIS METHOD NAME IS FIXED, you cannot override its NAME, but its BODY is exactly what your JSP markup translates into
6. `jspDestroy()` -> called ONCE, equivalent to a Servlet's `destroy()`

3. Diagram — What Actually Happens to Your .jsp File

```
hello.jsp hello_jsp.java hello_jsp.class
(what YOU write) -----> (auto-generated Servlet -----> (compiled, then
(1) SOURCE CODE, extends executed exactly
TRANSLATE HttpJspBase) (2) like any Servlet)
COMPILE
```

4. Real Life Analogy

- **Restaurant:** JSP is like writing a recipe card in mostly-plain-English cooking instructions, with just a FEW precise, standardized technical annotations sprinkled in ("preheat to 220°C") — the kitchen's OWN internal translation process converts this into the EXACT low-level machine instructions (the generated Servlet) that actually get executed, without you needing to write those low-level instructions by hand.

5. First Request vs Subsequent Requests

Request	What Happens
FIRST request to a JSP (or after the <code>.jsp</code> file changes)	Translation + Compilation occurs (noticeably SLOWER — this is the classic "JSP first-hit latency")
ALL SUBSEQUENT requests	The ALREADY-COMPILED Servlet class is reused directly — as fast as any regular Servlet

6. Edge Cases

Case	Result
A syntax error in the JSP markup	Fails at the TRANSLATION or COMPILATION step, producing a detailed error page (in development mode) showing the exact generated Java code and line
The <code>.jsp</code> file is modified after deployment	Most containers automatically detect the change and RE-translate/re-compile on the next request (development convenience; often disabled in production for performance)

7. Common Mistakes

- Assuming JSP is somehow fundamentally different from Servlets — it's translated INTO a Servlet; understanding this demystifies almost every JSP behavior by reducing it back to familiar Servlet lifecycle concepts.
- Not accounting for the "first-hit compilation delay" when load-testing/benchmarking a JSP-based application — the first request is genuinely, expectedly slower.

8. Frequently Asked Interview Questions

- What does a JSP file actually become at runtime?** It's translated into an equivalent Servlet Java source file, compiled, and then executed exactly like any other Servlet.
- What are the three key lifecycle methods of the generated Servlet, and their Servlet-lifecycle equivalents?** `jspInit()` ($\approx$ `init()`), `_jspService()` ($\approx$ `service()`), `jspDestroy()` ($\approx$ `destroy()`).
- Why is the FIRST request to a JSP page often noticeably slower than subsequent ones?** Because translation and compilation only happen on the first request (or after a file change) — subsequent requests reuse the already-compiled Servlet class directly.

TOPIC 2: JSP DIRECTIVES

1. Introduction

What is it? Directives are special JSP instructions, enclosed in `<%@ ... %>`, that configure how the JSP page itself is translated/processed — they don't produce output directly, but affect page-level settings (imports, error handling, included content).

2. The Three Directive Types

```
<%@ page import="java.util.List, java.util.ArrayList" contentType="text/html" %>
<%@ include file="header.jsp" %>
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
```

Directive	Purpose
page	Configures page-level attributes: <code>import</code> (Java imports), <code>contentType</code> , <code>errorPage</code> , <code>isErrorPage</code> , <code>session</code> (whether an implicit session object is available)
include	STATICALLY includes another file's content AT TRANSLATION TIME (merges source before compiling — like a textual copy-paste)
taglib	Declares a tag library (like JSTL, next-but-one topic) and its prefix, for use elsewhere in the page

3. page Directive — Common Attributes

```
<%@ page import="java.util.*, com.company.model.User" %>
<%@ page errorPage="error.jsp" %> <!-- redirects to error.jsp on any uncaught exception -->
<%@ page contentType="text/html; charset=UTF-8" %>
```

4. Static Include vs Dynamic Include — A Frequently Confused Pair

Aspect	<code><%@ include file="..." %></code> (directive)	<code><jsp:include page="..." /></code> (action)
Timing	TRANSLATION time — content is merged BEFORE compilation	REQUEST time — the included page is executed and its output merged at RUNTIME, on every request
Performance	Slightly faster (merged once)	Slightly slower (runtime dispatch every request), but more flexible
Use case	Static, rarely-changing content (headers/footers)	Dynamic content that may change per-request

5. Frequently Asked Interview Questions

- What are the three JSP directive types?** `page`, `include`, `taglib`.
- What's the difference between the `include` DIRECTIVE and the `jsp:include` ACTION?** The directive merges content STATICALLY at translation time (once); the action includes content DYNAMICALLY at request time (every request), allowing genuinely dynamic included content.
- What does the `page` directive's `errorPage` attribute do?** Specifies a JSP to redirect to automatically if an uncaught exception occurs during this page's processing.

TOPIC 3: SCRIPTLETS

1. Introduction

What is it? Scriptlets (`<% ... %>`) let you embed RAW JAVA CODE directly inside a JSP page — the original (and now heavily DISCOURAGED) way to add logic to a JSP, largely superseded by Expression Language (EL) and JSTL (next topics) for cleaner separation of concerns.

2. The Three Scripting Elements

```
<!-- private int counter = 0; %> <!-- DECLARATION - becomes a field in the generated Servlet -->

<% counter++; %> <!-- SCRIPTLET - becomes a statement inside _jspService() -->

<%= counter %> <!-- EXPRESSION - its value is directly PRINTED to the output -->
```

3. Why Scriptlets Are Discouraged (Critical Modern Understanding)

```
<!-- BAD - business logic and presentation tangled together, hard to test/maintain -->
<%
List<User> users = userService.getAllUsers();
for (User u : users) {
%>
<p><%= u.getName() %></p>
<%
}
%>
```

This kind of embedded Java control-flow logic directly inside HTML markup is:

- **Hard to read** — mixing languages/paradigms in one file.
- **Hard to test** — logic embedded in a JSP can't be unit-tested like a regular Java class/method.
- **A violation of MVC** (see [[MVC Architecture]], Advanced Java Topic 2) — the VIEW should be presentation-only, not contain business logic or complex control flow.

The modern replacement: JSTL (JSP Standard Tag Library, covered soon) provides tag-based equivalents (`<c:forEach>`, `<c:if>`) that achieve the same result WITHOUT embedding raw Java syntax in the markup.

4. Real Life Analogy

- **Restaurant:** Using scriptlets is like handing the WAITER (the View/JSP) a full, complex cooking recipe to execute tableside in front of customers, instead of having the KITCHEN (the Model/Controller) do all the actual cooking beforehand and just handing the waiter a simple, ready-to-serve plate — mixing responsibilities that should stay separate.

5. Frequently Asked Interview Questions

- 1 **What are the three JSP scripting elements?** Declaration (`<%! %>`), Scriptlet (`<% %>`), Expression (`<%= %>`).

- 2 **Why are scriptlets considered a bad practice in modern JSP development?** They embed raw Java logic directly in presentation markup, violating MVC separation of concerns, hurting readability, and making the logic untestable as a proper Java unit.
 - 3 **What modern alternatives replace scriptlets for control flow and logic?** JSTL tags (`<c:forEach>`, `<c:if>`) combined with Expression Language (EL) for displaying values.
-

TOPIC 4: EXPRESSION LANGUAGE (EL)

1. Introduction

What is it? Expression Language (`${...}`) provides a concise, Java-code-free way to ACCESS and DISPLAY data (request attributes, session attributes, JavaBean properties) directly within JSP markup — the standard, modern replacement for scriptlet expressions (`<%= %>`).

2. Syntax and Scope Resolution

```
${user.name} <!-- accesses user.getName() - EL automatically calls the GETTER -->
${param.username} <!-- accesses a REQUEST PARAMETER (query string / form field) directly -->
${sessionScope.cartTotal} <!-- explicitly scoped to SESSION attributes -->
${requestScope.errorMessage} <!-- explicitly scoped to REQUEST attributes -->
```

- Without an explicit scope prefix (`${name}`), EL searches PAGE → REQUEST → SESSION → APPLICATION scope, in that order, using the FIRST match found.
- EL automatically calls the appropriate GETTER method for JavaBean-style property access (`user.name` → `user.getName()`).

3. EL Operators

```
${1 + 2} <!-- arithmetic: 3 -->
${user.age >= 18 ? "Adult" : "Minor"} <!-- ternary conditional -->
${empty cartItems} <!-- checks for null OR empty collection/string -->
${user != null && user.active} <!-- logical operators -->
```

4. Code Example

```
<%-- Assume the Controller did: request.setAttribute("user", someUserObject); --%>
<html>
<body>
<h1>Welcome, ${user.name}!</h1>
<p>Your account balance is: ${user.balance}</p>
<p>Status: ${user.balance > 0 ? "Active" : "Overdrawn"}</p>
</body>
</html>
```

5. `${}` (EL, JSP standard) vs `#{}` (JSF-specific) — A Common Confusion

`${}` is standard JSP Expression Language, evaluated once at REQUEST time. `#{}` is a DIFFERENT syntax used specifically by JavaServer Faces (JSF, a separate, less commonly used Java EE component-based UI framework) supporting deferred/bidirectional evaluation — NOT interchangeable with plain JSP's `${}`.

6. Edge Cases

Case	Result
<code>\${user.name}</code> where <code>user</code> is <code>null</code>	Evaluates to an EMPTY STRING, NOT an exception (a deliberate EL design choice — much friendlier for templating than throwing <code>NullPointerException</code>)
Accessing a NON-EXISTENT attribute	Also evaluates to empty string / nothing, no exception
A getter method THROWS an exception when EL tries to call it	Propagates as a genuine error, unlike simple null/absent-attribute cases

7. Common Mistakes

- Confusing EL's `${}` with JSF's `#{}` syntax.
- Forgetting EL requires proper JavaBean-style GETTER methods (`getName()`) on the underlying object — EL cannot access `public` fields directly by convention, only via getters.
- Overcomplicating EL expressions with deeply nested logic — if an expression becomes complex, that logic likely belongs in the Controller/Model instead (same principle as avoiding scriptlets).

8. Best Practices

- Use EL for ALL simple data display in JSP — it's the modern standard, replacing scriptlet expressions entirely.
- Keep EL expressions SIMPLE (direct property access, simple conditionals) — genuinely complex logic belongs in Java code (Controller/Service), not embedded in the View.

9. Frequently Asked Interview Questions

- 1 **What is Expression Language (EL) used for?** Concisely accessing and displaying data (bean properties, request/session attributes) in JSP markup without writing raw Java code.
- 2 **What happens if you access a property of a `null` object via EL, e.g., `${user.name}` where `user` is `null`?** It evaluates to an empty string, NOT a `NullPointerException` — a deliberate design choice for template-friendliness.
- 3 **What scope resolution order does EL use for `${name}` with no explicit scope prefix?** Page → Request → Session → Application, using the first match found.
- 4 **Is `${}` the same as `#{}` ?** No — `${}` is standard JSP EL; `#{}` is JSF-specific syntax supporting deferred evaluation, used in a different framework context.

TOPIC 5: JSTL (JSP STANDARD TAG LIBRARY)

1. Introduction

What is it? JSTL is a standard collection of custom JSP tags providing common templating functionality — loops, conditionals, formatting — WITHOUT writing any raw Java scriptlet code, completing the modern replacement for scriptlets (EL handles DATA DISPLAY, JSTL handles CONTROL FLOW/LOGIC-LIKE STRUCTURE).

2. The Core (c) Tag Library — Most Common Tags

```
<%@ taglib uri="jakarta.tags.core" prefix="c" %>

<c:if test="${user.age >= 18}">
<p>Welcome, adult user!</p>
</c:if>

<c:choose>
<c:when test="${user.balance > 1000}">
<p>Premium customer</p>
</c:when>
<c:otherwise>
<p>Standard customer</p>
</c:otherwise>
</c:choose>

<c:forEach var="item" items="${cartItems}">
<p>${item.name} - ${item.price}</p>
</c:forEach>

<c:set var="total" value="${100 + 50}" />
<c:out value="${total}" />
```

3. Comparison — Scriptlet Loop vs JSTL Loop (Direct Before/After)

```
<!-- OLD WAY (scriptlet - discouraged): -->
<%
for (User u : userList) {
%>
<p><%= u.getName() %></p>
<%
}
%>

<!-- MODERN WAY (JSTL + EL): -->
<c:forEach var="u" items="${userList}">
<p>${u.name}</p>
</c:forEach>
```

The JSTL version is more readable, keeps the markup looking like actual HTML+tags rather than a mix of Java and HTML, and (importantly) STILL avoids any raw Java syntax in the View — maintaining the MVC separation-of-concerns principle.

4. Other JSTL Libraries (Beyond Core)

Library	Prefix	Purpose
Core	<code>c</code>	Control flow, variables (<code>if</code> , <code>forEach</code> , <code>choose</code> , <code>set</code>)
Formatting	<code>fmt</code>	Number/date formatting, internationalization
SQL	<code>sql</code>	Direct database queries from JSP (STRONGLY discouraged in modern architecture — violates MVC by putting data access in the View)
XML	<code>x</code>	XML processing (rarely used today)
Functions	<code>fn</code>	String utility functions (<code>fn:length</code> , <code>fn:toUpperCase</code>)

5. Real Life Analogy

- **Restaurant:** JSTL is like giving the waiter (View) a standardized set of pre-approved SERVICE GESTURES ("if the customer orders dessert, bring the dessert menu"; "for each item on this order, print a line on the receipt") instead of asking the waiter to write and execute custom low-level cooking code themselves — clean, safe, standardized actions instead of raw programming logic in the presentation layer.

6. Edge Cases

Case	Result
Using <code><sql:query></code> to run database queries directly in a JSP	Technically works, but is a WELL-KNOWN anti-pattern — mixes data access into the View, violating MVC; virtually never used in well-architected modern applications
<code><c:forEach></code> over a <code>null</code> collection	Simply produces no output/iterations (no exception), consistent with EL's null-friendly philosophy

7. Common Mistakes

- Using the `sql` tag library for actual data access in production applications — a serious MVC/architecture violation; database access belongs in the Model/Service layer, never the View.
- Reverting to scriptlets "just this once" for a slightly complex case instead of finding the appropriate JSTL tag or moving the logic to the Controller.

8. Best Practices

- Use JSTL Core (`c:if`, `c:forEach`, `c:choose`) combined with EL for essentially ALL JSP logic/display needs.
- NEVER use the `sql` tag library in real applications — it exists mostly for historical/educational completeness, not genuine architectural recommendation.

- If JSTL + EL genuinely cannot express something cleanly, that's usually a signal the logic belongs in the Controller/Model, not a reason to fall back to scriptlets.

9. Frequently Asked Interview Questions

- 1 **What is JSTL?** A standard library of custom JSP tags providing loops, conditionals, and formatting without writing raw Java scriptlet code.
- 2 **What's the relationship between EL and JSTL?** EL handles DISPLAYING data (accessing properties/values); JSTL handles CONTROL FLOW and logic-like structure (loops, conditionals) — together they fully replace the need for scriptlets.
- 3 **Why is the JSTL `sql` tag library considered an anti-pattern?** It performs database access directly from the View, violating MVC's separation of concerns — data access belongs in the Model/Service layer.
- 4 **Rewrite a scriptlet-based for-loop over a `List<User>` using JSTL.** `<c:forEach var="u" items="${userList}"><p>${u.name}</p></c:forEach>`.

TOPIC 6: CUSTOM TAGS

1. Introduction

What is it? Beyond the STANDARD JSTL tags, JSP allows developers to define their OWN custom tags — reusable, encapsulated presentation logic packaged as a tag, backed by a Java class (a "Tag Handler") implementing specific tag-handling interfaces.

2. Building a Simple Custom Tag

```
// Tag Handler class
package com.company.tags;

import jakarta.servlet.jsp.tagext.*;
import java.io.IOException;

public class GreetingTag extends TagSupport {
    private String name;

    public void setName(String name) { this.name = name; } // corresponds to a tag ATTRIBUTE

    @Override
    public int doStartTag() throws JspException {
        try {
            pageContext.getOut().print("Hello, " + name + "!");
        } catch (IOException e) {
            throw new JspException(e);
        }
        return SKIP_BODY;
    }
}
```

```

<!-- Tag Library Descriptor: greeting.tld -->
<taglib>
<tag>
<name>greet</name>
<tag-class>com.company.tags.GreetingTag</tag-class>
<attribute>
<name>name</name>
<required>true</required>
</attribute>
</tag>
</taglib>

<%@ taglib uri="/WEB-INF/greeting.tld" prefix="my" %>
<my:greet name="Alice" /> <!-- outputs: Hello, Alice! -->

```

3. Real Life Analogy

- **Restaurant:** A custom tag is like the restaurant chain inventing its OWN standardized internal shorthand instruction ("Prepare Table Special #3") that any location's staff can use, encapsulating a specific sequence of steps behind one simple, reusable command — just like JSTL's built-in tags, but custom-defined for YOUR specific application's needs.

4. Custom Tags vs JSTL — When to Build Your Own

Scenario	Recommendation
Common looping/conditional logic	Use STANDARD JSTL (<code>c:forEach</code> , <code>c:if</code>) — no need to reinvent
Genuinely reusable, application-specific presentation logic (e.g., a standardized "format currency with company branding" display)	A custom tag CAN be justified
Modern applications	Custom tags have become quite rare — most new development uses Thymeleaf (Spring Boot's preferred templating engine, covered later) or a separate frontend framework (React) instead of JSP custom tags entirely

5. Frequently Asked Interview Questions

- 1 **What is a custom JSP tag?** A reusable, encapsulated piece of presentation logic, defined as a tag, backed by a Java "Tag Handler" class.
- 2 **What file declares a custom tag's name and attributes?** A Tag Library Descriptor (`.tld` file).
- 3 **Are custom JSP tags commonly used in modern development?** Rarely — most new projects use Thymeleaf or a separate frontend framework (React/Angular) instead of building custom JSP tags.

TOPIC 7: MVC WITH JSP

1. Introduction

What is it? This topic ties together everything in this section — showing how JSP fits as the VIEW component in the classic MVC pattern (fully introduced in Advanced Java, Topic 2), commonly called the **"Model 2" architecture** in classic Java EE terminology (as opposed to "Model 1," an older, now-discouraged pattern where JSP handled EVERYTHING itself, including business logic and control flow).

2. Model 1 vs Model 2 Architecture

```
MODEL 1 (discouraged, historical):
Browser -> JSP (handles EVERYTHING: business logic, data access, AND presentation)

MODEL 2 (MVC, the standard approach):
Browser -> Servlet/Controller -> Model (business logic/data access)
|
v
JSP (View, PURELY presentation, using EL + JSTL only)
```

3. Full Code Example — A Complete Model 2 (MVC) Flow

```
// MODEL
class Product {
String name;
double price;
Product(String name, double price) { this.name = name; this.price = price; }
// getters...
}

// CONTROLLER
@WebServlet("/products")
public class ProductController extends HttpServlet {
protected void doGet(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
List<Product> products = productService.findAll(); // MODEL logic (delegated)
request.setAttribute("products", products); // pass data to the VIEW
request.getRequestDispatcher("/WEB-INF/views/productList.jsp")
.forward(request, response); // forward, NOT redirect (Servlets Topic 4)
}
}

<!-- VIEW: /WEB-INF/views/productList.jsp - PURE presentation, EL + JSTL only, ZERO scriptlets --%>
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
<html>
<body>
<h1>Product List</h1>
<c:forEach var="p" items="${products}">
<p>${p.name} - ${p.price}</p>
</c:forEach>
</body>
</html>
```

Notice the `.jsp` file is placed inside `WEB-INF/views/` — a deliberate best practice, since `WEB-INF/` contents are NEVER directly browser-accessible (see [\[\[Web Application Basics\]\]](#), Advanced Java Topic 3), FORCING all requests to go through the Controller first, rather than allowing direct access to a raw View.

4. Real Life Analogy

- **Restaurant:** Model 2/MVC-with-JSP is like ensuring EVERY customer order goes through the proper front-of-house waiter (Controller) who coordinates with the kitchen (Model) before ANY plated dish (View/JSP) reaches a table — customers are never allowed to walk directly into the kitchen and grab food themselves (direct JSP access bypassing the Controller).

5. Edge Cases

Case	Result
A JSP View placed OUTSIDE WEB-INF/	Directly browser-accessible via URL, bypassing the intended Controller entirely — a common security/architecture mistake
A JSP View placed INSIDE WEB-INF/	Only reachable via <code>forward()</code> from a Controller — enforces the intended MVC flow

6. Common Mistakes

- Placing View JSPs outside WEB-INF/, allowing users to bypass the Controller and access "raw" views directly (potentially without proper data/authorization checks having been performed).
- Falling back to Model 1-style JSPs (embedding business logic directly) for "just one quick page" — undermines the entire architecture's maintainability benefits.

7. Best Practices

- Place all View JSPs inside WEB-INF/ (or a subdirectory) specifically to force all access through the Controller.
- Strictly maintain the Model 2 (MVC) pattern — Controllers handle requests and delegate to Models, Views (JSP + EL + JSTL) handle ONLY presentation.

8. Production Usage

- Classic, still-maintained enterprise Java web applications (many built years ago and still running) commonly use exactly this Servlet + JSP Model 2 pattern.
- Modern Spring MVC (covered in depth later in this course) is CONCEPTUALLY IDENTICAL to this pattern — `@Controller` classes replace raw Servlets, and Thymeleaf templates (or still JSP, supported but less common now) replace hand-written JSP views — the architecture you've learned here transfers directly.

9. Frequently Asked Interview Questions

- 1 **What is "Model 2" architecture?** The standard MVC approach for JSP-based applications — a Servlet/Controller handles requests and business logic delegation, forwarding to a JSP View that handles ONLY presentation.

- 2 **What is "Model 1," and why is it discouraged?** An older pattern where a single JSP handles EVERYTHING (business logic, data access, AND presentation) — discouraged because it violates separation of concerns and becomes unmaintainable at scale.
- 3 **Why should View JSPs be placed inside WEB-INF/?** Because WEB-INF/ contents are never directly browser-accessible, forcing all requests to go through the Controller first, rather than allowing users to bypass it and access raw views directly.
- 4 **How does Model 2/MVC-with-JSP relate to modern Spring MVC?** It's conceptually identical — Spring's @Controller classes replace raw Servlets, and templating engines (Thymeleaf, or still JSP) replace hand-written views, but the Controller → Model → View flow is the same pattern.

10. Revision Notes (Entire JSP Section Consolidated)

```
JSP - CHEAT SHEET
=====
JSP = translated into a Servlet at first request (Translation -> Compilation ->
jspInit() -> _jspService() [per request] -> jspDestroy())

DIRECTIVES: <%@ page/include/taglib %> - configure the page, NOT output directly
include DIRECTIVE = translation-time merge; jsp:include ACTION = runtime merge

SCRIPTLETS (<% %>, <%= %>, <%! %>): raw Java in markup - DISCOURAGED, replaced by:
EL (${...}) -> DISPLAYS data, null-safe (empty string, not NPE)
JSTL (<c:...>) -> CONTROL FLOW (if/forEach/choose), no raw Java needed

CUSTOM TAGS: your own reusable tags (Tag Handler class + .tld) - rare in modern dev

MVC WITH JSP (Model 2): Servlet/Controller -> Model -> JSP View (EL+JSTL only)
View JSPs go in WEB-INF/ to FORCE requests through the Controller first
(Model 1 = JSP does everything - DEPRECATED anti-pattern)

MODERN CONTEXT: this ENTIRE pattern maps directly onto Spring MVC + Thymeleaf later
```

11. Homework (Covers Entire JSP Section)

- 1 Build a simple Model 2 application: a Servlet Controller fetching a List<Product>, forwarding to a WEB-INF/-placed JSP View using ONLY EL + JSTL (zero scriptlets) to display it.
- 2 Rewrite a scriptlet-based conditional (<% if (...) { %>) using <c:if> and EL instead.
- 3 Explain, in your own words, why placing a View JSP inside WEB-INF/ improves the application's architecture.

FINAL SUMMARY — JSP COMPLETE

This concludes the **JSP** section — 7 topics:

- 1 **JSP Lifecycle** — translation → compilation → jspInit()/_jspService()/jspDestroy(), the first-hit compilation delay.
- 2 **Directives** — page, include, taglib; static vs dynamic includes.
- 3 **Scriptlets** — the old, discouraged way to embed Java logic; why it violates MVC.

- 4 **Expression Language (EL)** — `${...}` for null-safe data display, scope resolution.
- 5 **JSTL** — `<c:forEach>`, `<c:if>`, `<c:choose>` as the modern, scriptlet-free control-flow replacement.
- 6 **Custom Tags** — building your own reusable tag handlers (now rare in modern development).
- 7 **MVC with JSP** — Model 2 architecture, the Servlet-Controller → Model → JSP-View flow, and its direct conceptual mapping onto modern Spring MVC.

You now fully understand the classic Java web stack (Servlets + JSP + MVC) that modern Spring Boot/Spring MVC directly evolved from — every concept here (Controllers, Views, request/session scope, MVC separation) reappears, just with less boilerplate, in the Spring sections ahead.

(END OF JSP MASTER NOTES — ready to proceed to SQL whenever you say "Next Topic.")